Introduction to Java Servlets, JSPs, and MVC

Ehsan Khamespanah

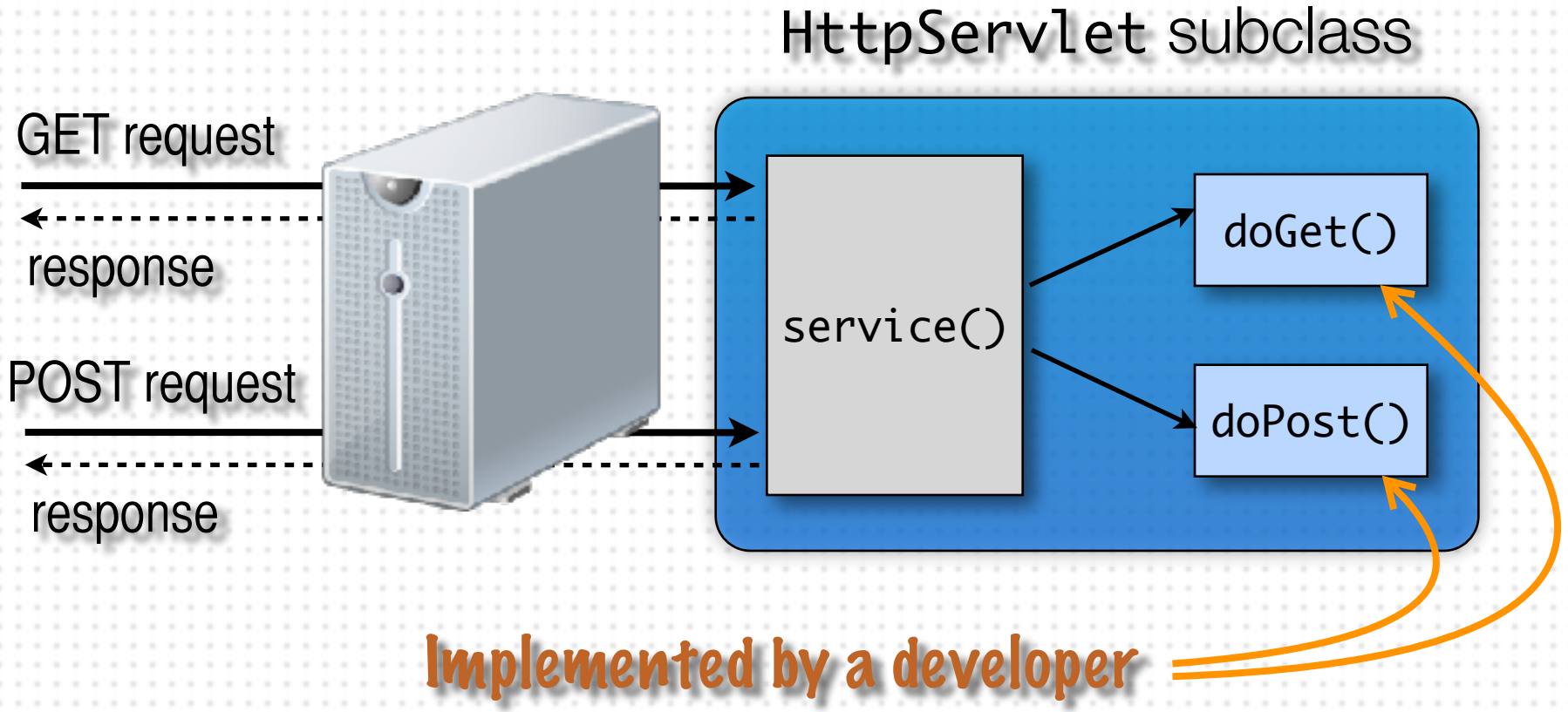
Spring 2026

Java Servlets

Wiki: A Java servlet is a Java software component that extends the capabilities of a server. They commonly implement web containers for hosting web applications on web servers and thus qualify as a server-side servlet web API.

- Generic Servlets
  - Must extend javax.servlet.Servlet
  - Protocol-independent
- HTTP Servlets
  - Must extend javax.servlet.http.HttpServlet
  - Handling HTTP request/reply

HTTP Servlets



An Example of HTTP Servlet Class

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloWorld extends HttpServlet {
  public void doGet(
      HttpServletRequest request,
      HttpServletResponse response)
      throws ServletException, IOException {

    // body of doGet

  }
}
```

An Example of HTTP Servlet Class

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloWorld extends HttpServlet {
  public void doGet(
      HttpServletRequest
      HttpServletRespons
      throws ServletExce

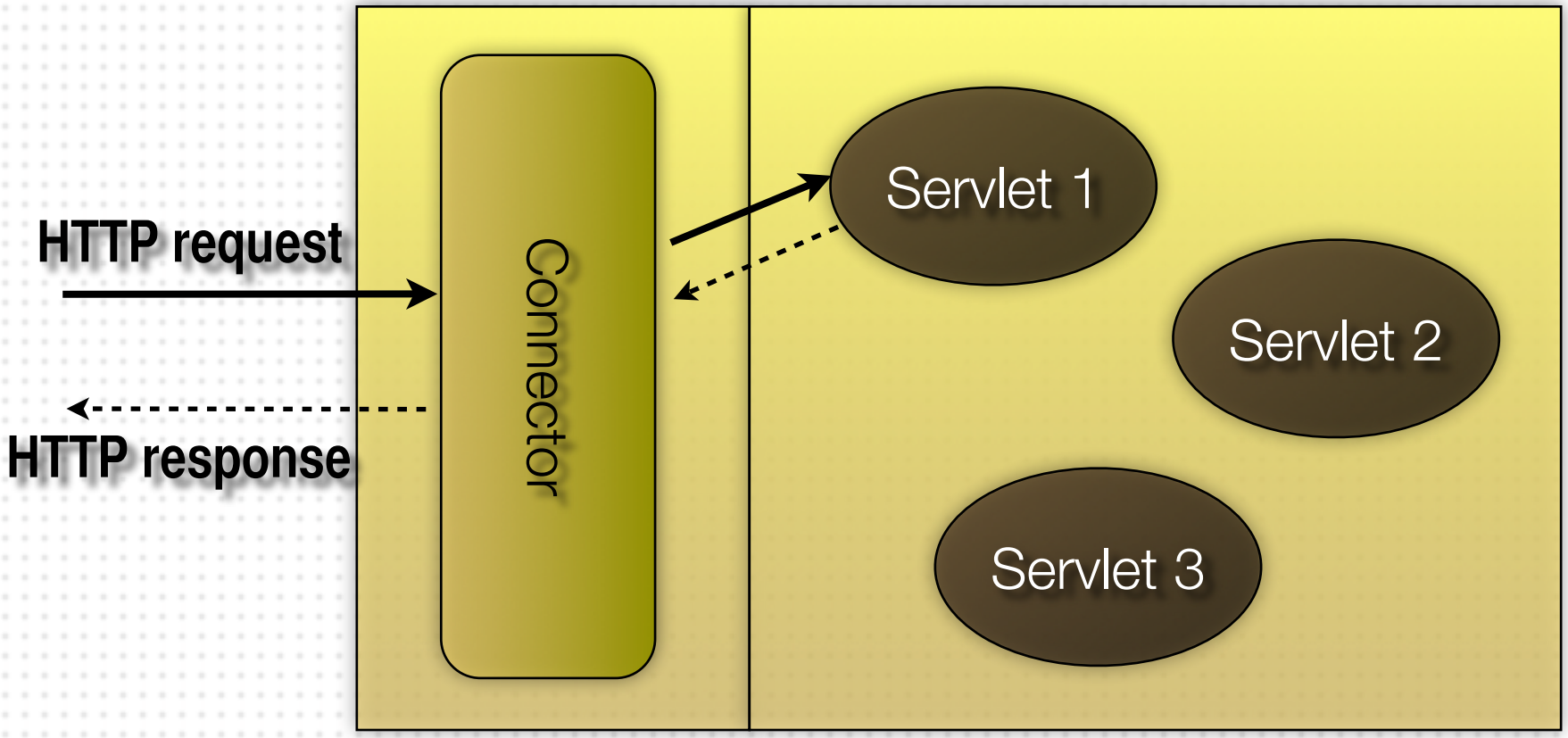
    // body of doGet

  }
}
```

```
response.setContentType("text/html");
PrintWriter out = response.getWriter();

out.println("<HTML>");
out.println("<HEAD>");
out.println("<TITLE>Hello World</TITLE>");
out.println("</HEAD>");
out.println("<BODY>");
out.println("<H1>Hello World</H1>");
out.println("</BODY></HTML>");
```

Web Containers



Some of a Web Container Responsibilities

- Parse the incoming HTTP request
- Determine which servlet must be run
- Load the servlet class (if not already loaded)
- Make an instance of the servlet (if not already done)
- Call doXYZ method (according to the HTTP method)
- Produce the HTTP servlet response

Some of a Web Container Responsibilities

- Parse the incoming HTTP request
- Determine which servlet must be run
- Load the servlet class (if not already loaded)
- Make an instance of the servlet (if not already done)
- Call doXYZ method (according to the HTTP method)
- Produce the HTTP servlet response

using reflection

J2EE Implementations

- Different vendors implement different part of the standard



Apache Tomcat

jetty://

ORACLE

WebLogic

IBM WebSphere

Project GlassFish

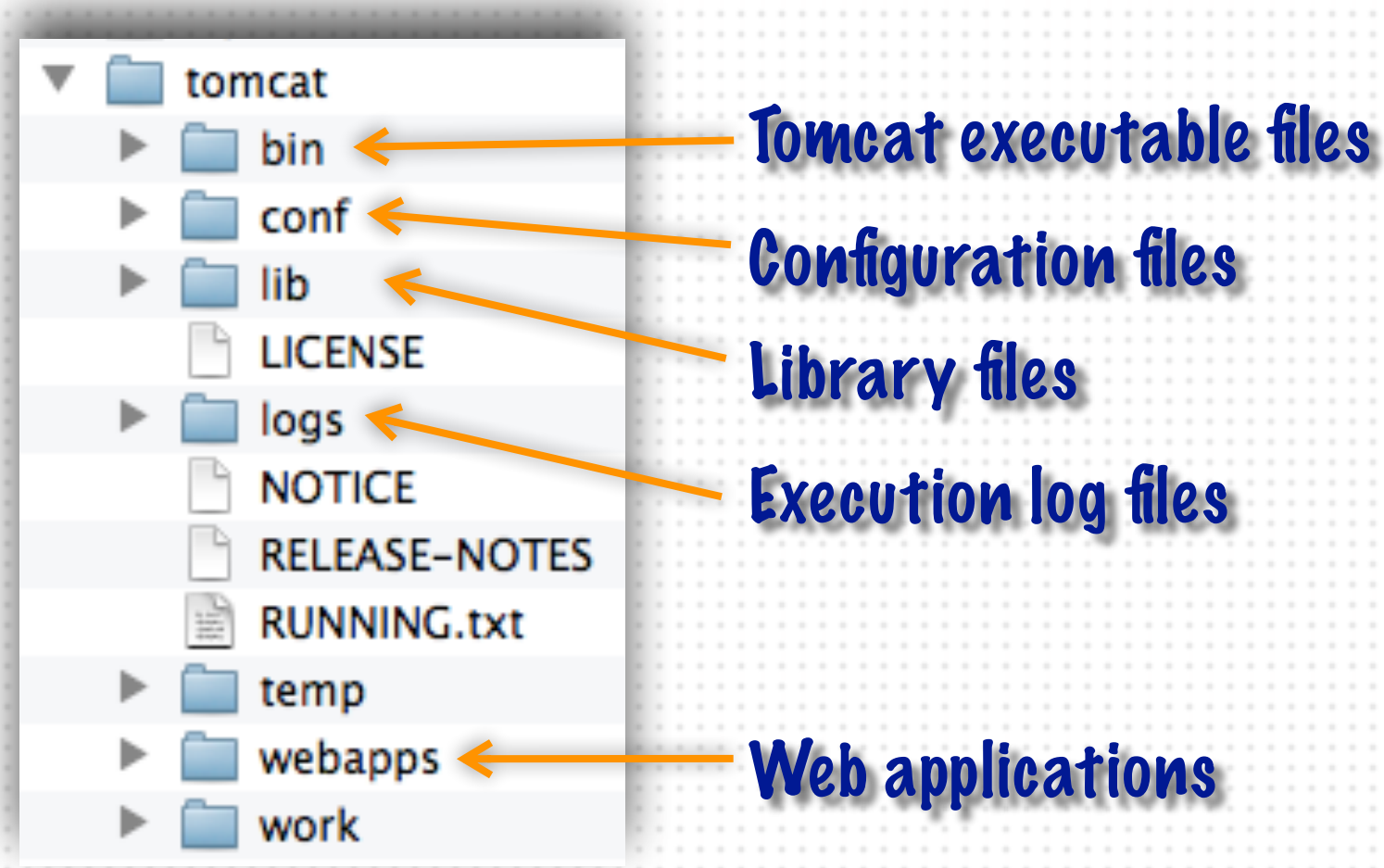


JBoss a division of Red Hat

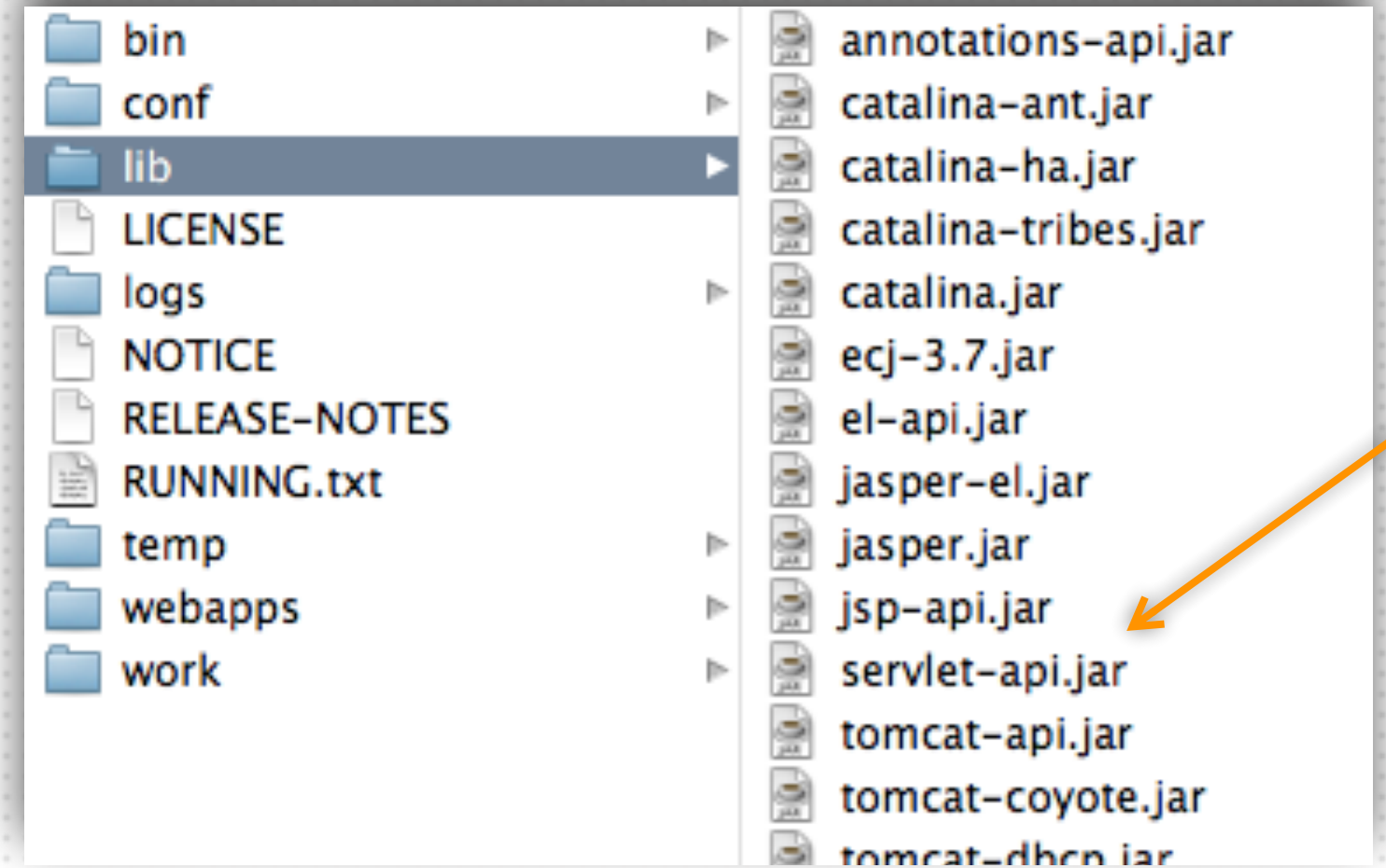
JBoss Application Server

Tomcat Directory Structure

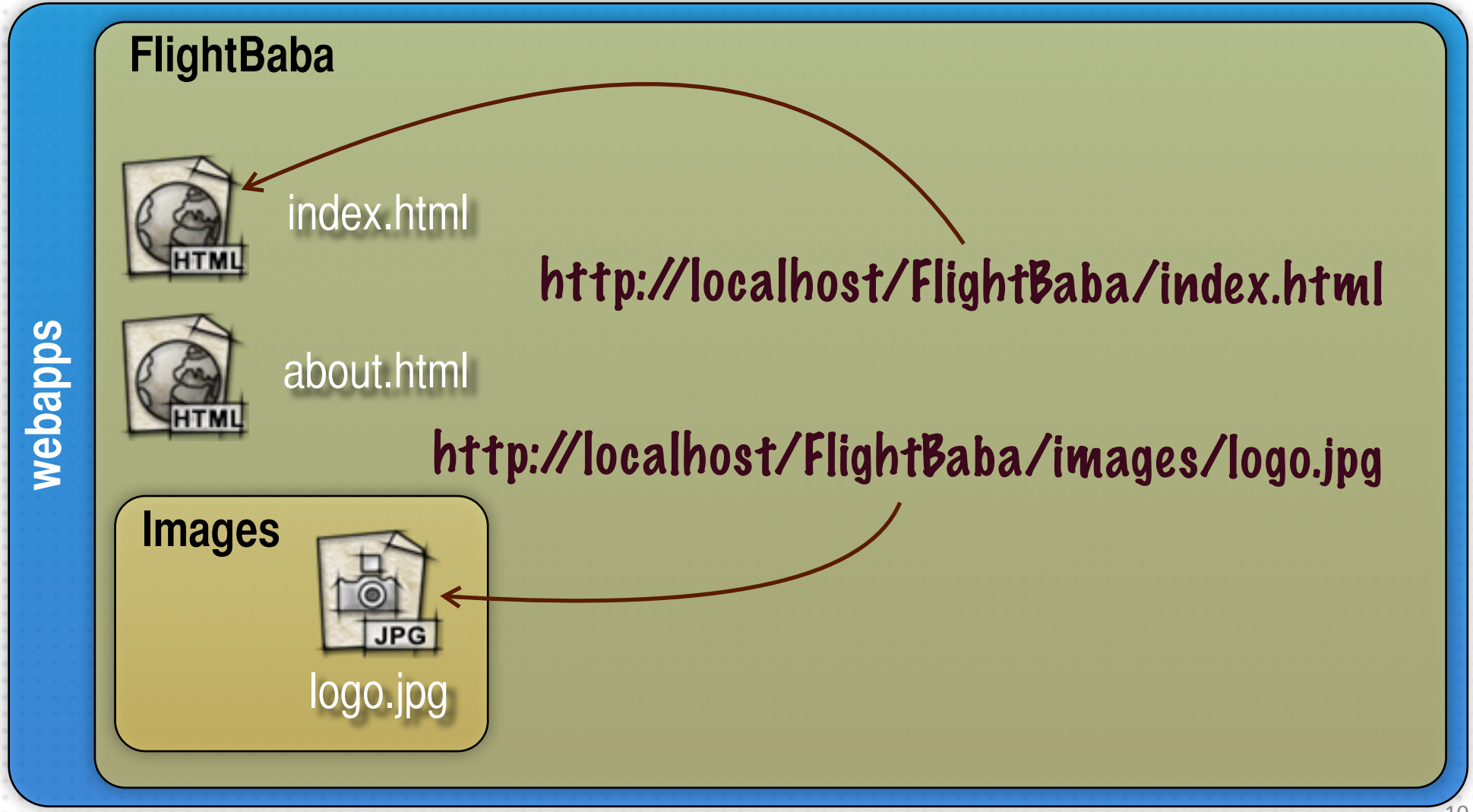




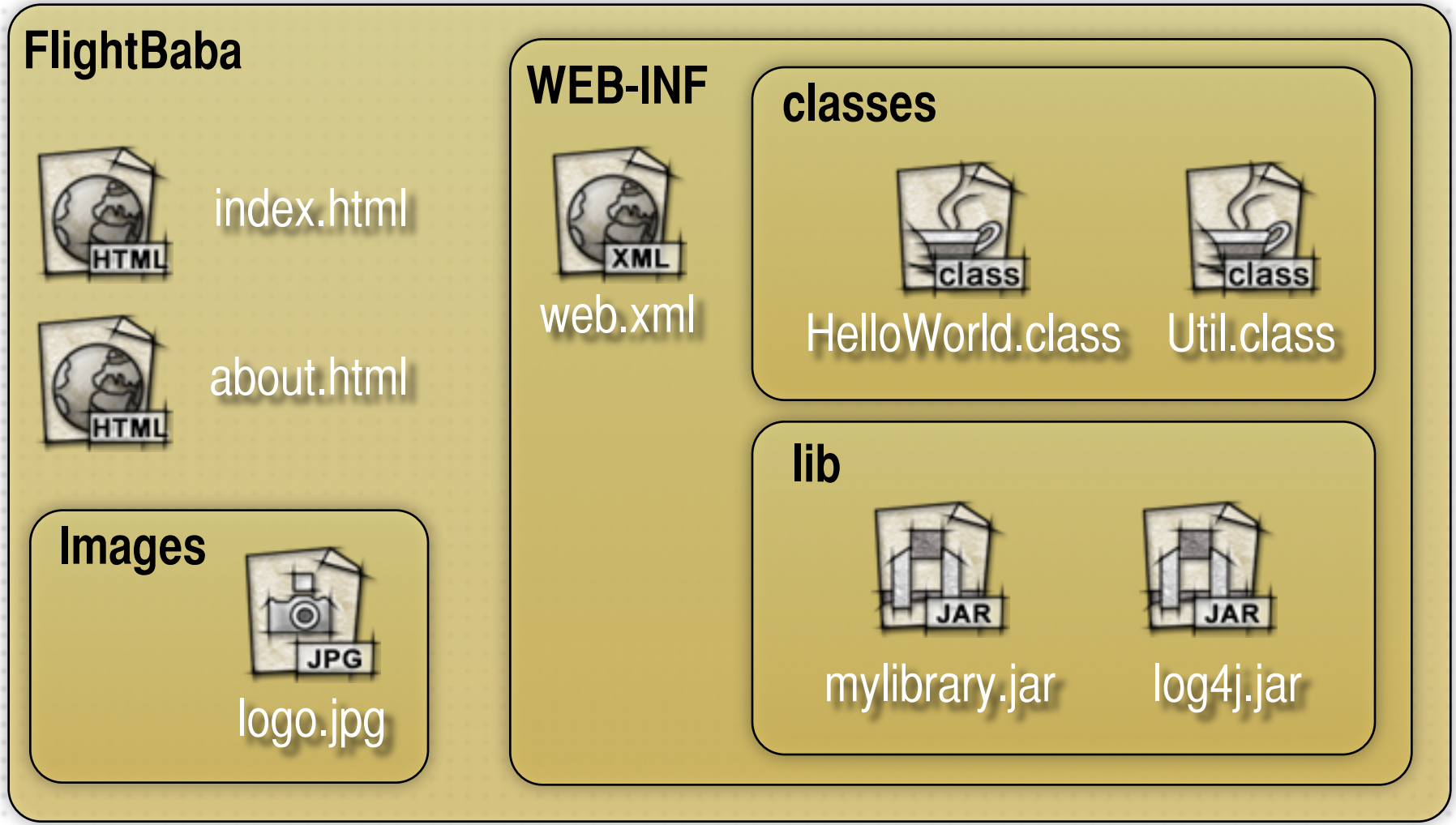
Tomcat Implementation



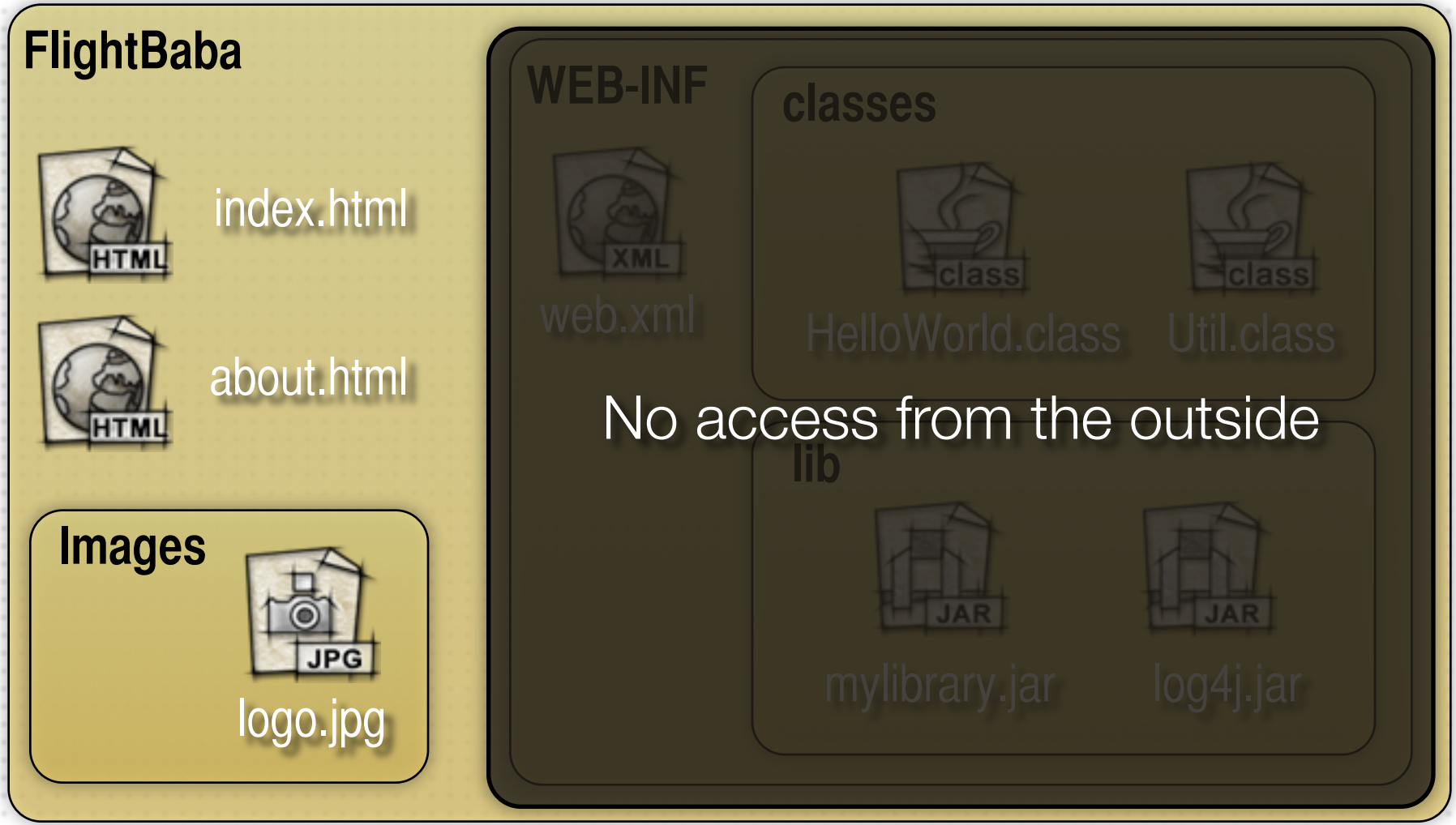
Web Apps - Static Content



Web Apps - Dynamic Content



Web Apps - Dynamic Content



Deployment Descriptor

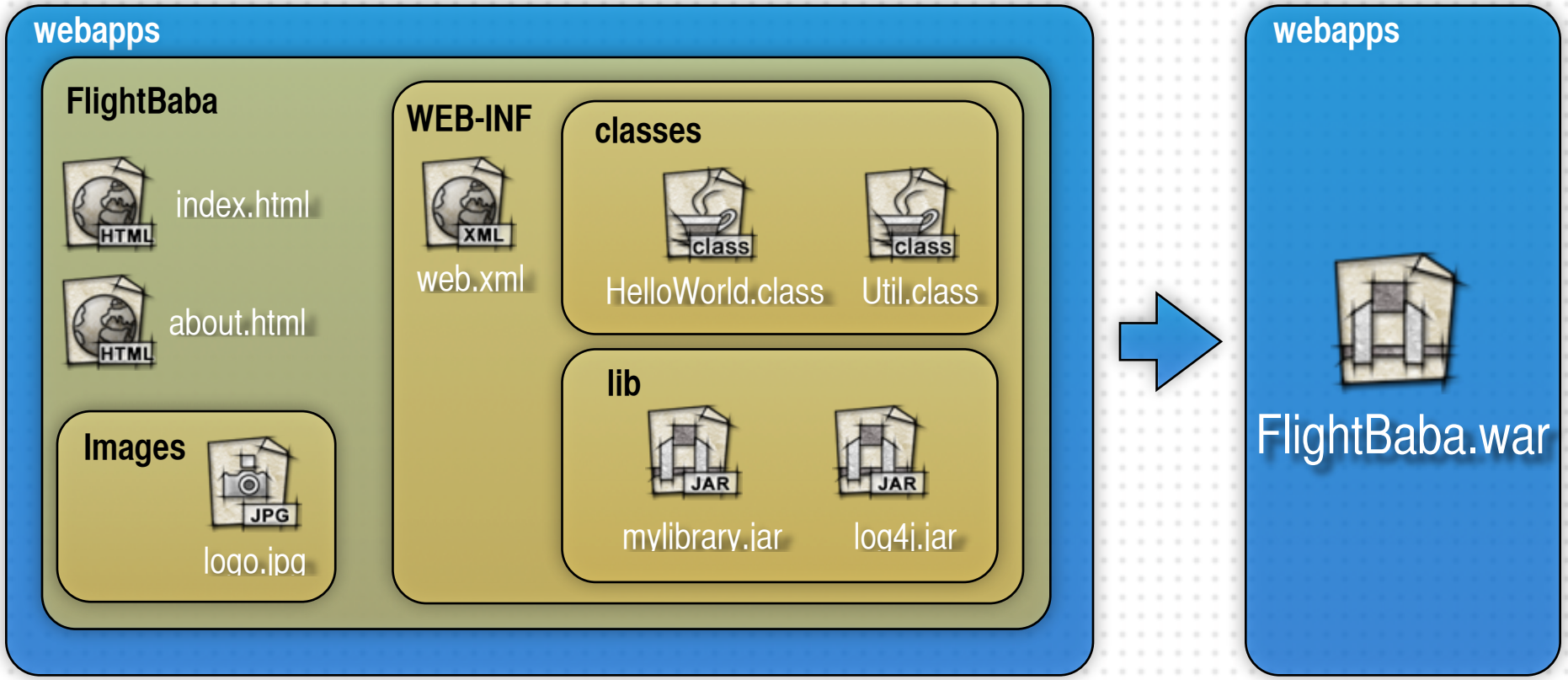
```
<web-app>

  <servlet>
    <servlet-name>HelloServlet</servlet-name>
    <servlet-class>ir.ac.ut.ece.ie.Hello</servlet-class>
  </servlet>

  <servlet-mapping>
    <servlet-name>HelloServlet</servlet-name>
    <url-pattern>/sayhello</url-pattern>
  </servlet-mapping>

</web-app>
```

Web Archives



Using Annotations

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;

@WebServlet("/hello")
public class HelloWorld extends HttpServlet {
    public void doGet(...) throws ... {
        // doGet body
    }
}
```

Difficulties

- It is hard to take care of output HTML tags
- Java and HTML are mixed together
  - Java developers like to develop logic
  - UI designers like to work with HTML tags
- Pages are many HTML tags with few java codes

Java Server Pages (JSP)

- Java Server Pages are HTML pages embedded with snippets of Java code.
  - It is an inverse of a Java Servlet
- Four different elements are used in constructing JSPs
  - Scripting Elements
  - Implicit Objects
  - Directives
  - Actions

Java Server Pages (JSP)

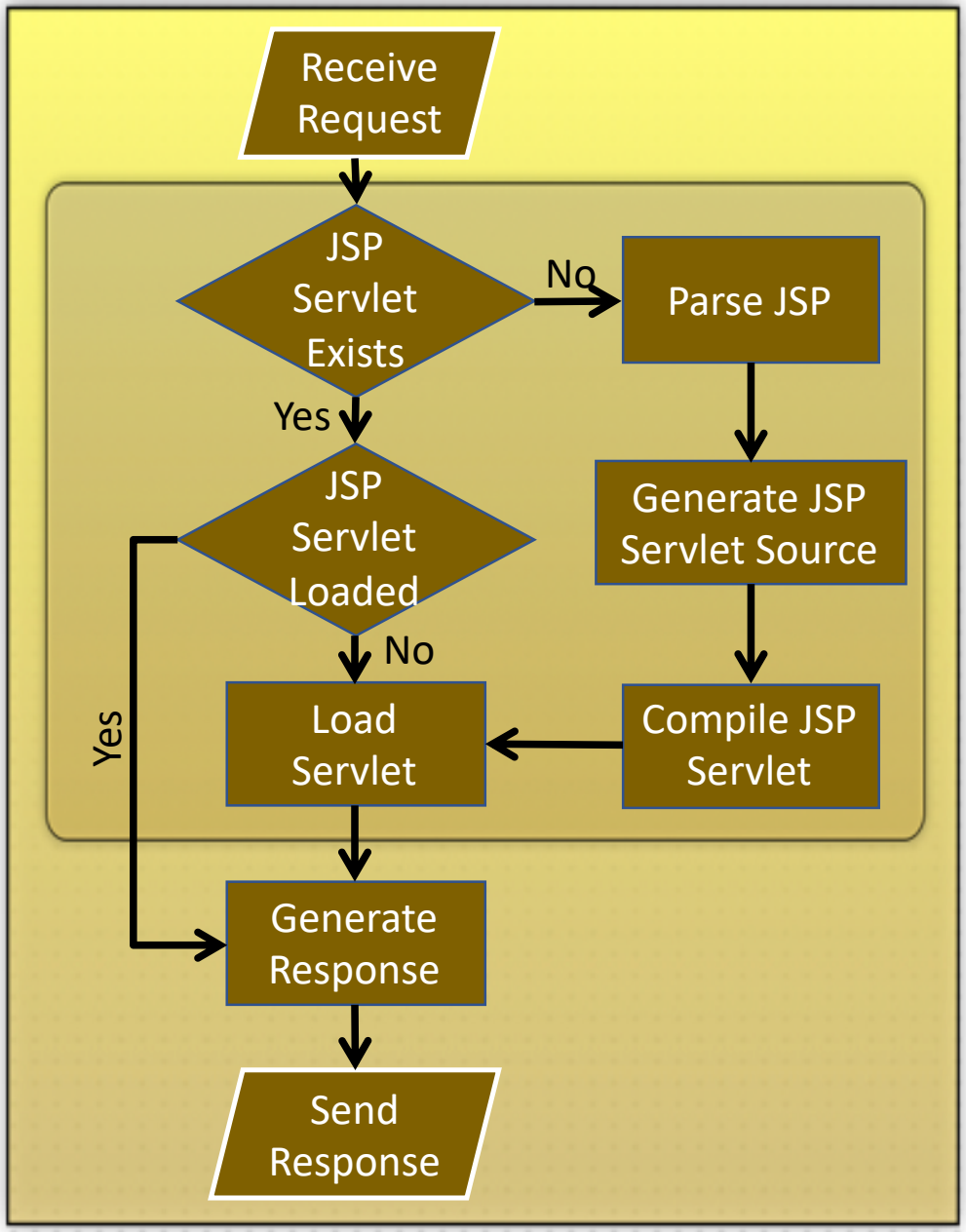
- Java Server Pages are HTML pages embedded with snippets of Java

```
<%@ page language="java" contentType="text/html; charset=US-ASCII"
    pageEncoding="US-ASCII"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "https://www.w3.org">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=US-ASCII">
<title>First JSP</title>
</head>
<%@ page import="java.util.Date" %>
<body>
<h3>Hi Pankaj</h3><br>
<strong>Current Time is</strong>: <%=new Date() %>

</body>
</html>
```

How to Handle JSPs

- JSPs run in two phases
  - Translation Phase
  - Execution Phase
- In translation phase JSP page is compiled into a servlet
- In execution phase the compiled JSP is processed



Scripting Elements

- **Declarations:** are used to define methods & instance variables
  - Do not produce any output that is sent to client
  - Embedded in <%! and %> delimiters

Scripting Elements

- **Declarations:** are used to define methods & instance variables
  - Do not produce any output that is sent to client
  - Embed

```
<%!
  Public void jspDestroy() {
    System.out.println("JSP Destroyed");
  }
  Public void jspInit() {
    System.out.println("JSP Loaded");
  }
  int myVar = 123;
%>
```

Scripting Elements

- **Declarations:** are used to define methods & instance variables
  - Do not produce any output that is sent to client
  - Embedded in <%! and %> delimiters
- **Scriptlets:** Used to embed java code in JSP pages
  - Embedded in <% %> delimiters

Scripting Elements

- **Declarations:** are used to define methods & instance variables
  - Do not produce any output that is sent to client
  - Embedded in <%! and %> delimiters
- **Scriptlets:** Used to embed java code in JSP pages
  - Embedded in <% %> de

```
<%
   int x = 5;
   int y = 7;
   int z = x + y;
%>
```

Scripting Elements

- **Declarations:** are used to define methods & instance variables
  - Do not produce any output that is sent to client
  - Embedded in <%! and %> delimiters
- **Scriptlets:** Used to embed java code in JSP pages
  - Embedded in <% %> delimiters
- **Expressions:** Used to write dynamic content back to the browser
  - The value of Java primitive or the result of toString is printed back to the browser
  - Embedded in <%= and %> delimiters

Scripting Elements

- **Declarations:** are used to define methods & instance variables
  - Do not produce any output that is sent to client
  - Embedded in <%! and %> delimiters
- **Scriptlets:** Used to embed java code in JSP pages
  - Embedded in <% %> delimiters
- **Expressions:** Used to write dynamic content back to the browser
  - The value of Java primi d back to the browser
  - Embedded in <%= and %

```
<%="Fred" + " " + "Flintstone"%>

<%=Math.sqrt(100)%>
```

Implicite Objects

- Implicit objects provide access to server side objects
  - **request:** Reference to the current request
  - **response:** Response to the request
  - **session:** session associated with current request
  - **config:** Servlet configuration for the page
  - **page:** instance of the page implementation class (this)
  - **exception:** Available with JSP pages which are error pages

Directives

- Messages sent to the JSP container: Aids the container in page translation
- Used for: Import required classes, Set output buffering options, Include content from external files, …

```
<%@
  page language = "java"
  buffer = "10kb"
  errorPage = “/error.jsp"
  import = “java.util.*, javax.sql.RowSet”
%>
<%@ include file = "file.jsp" %>
```

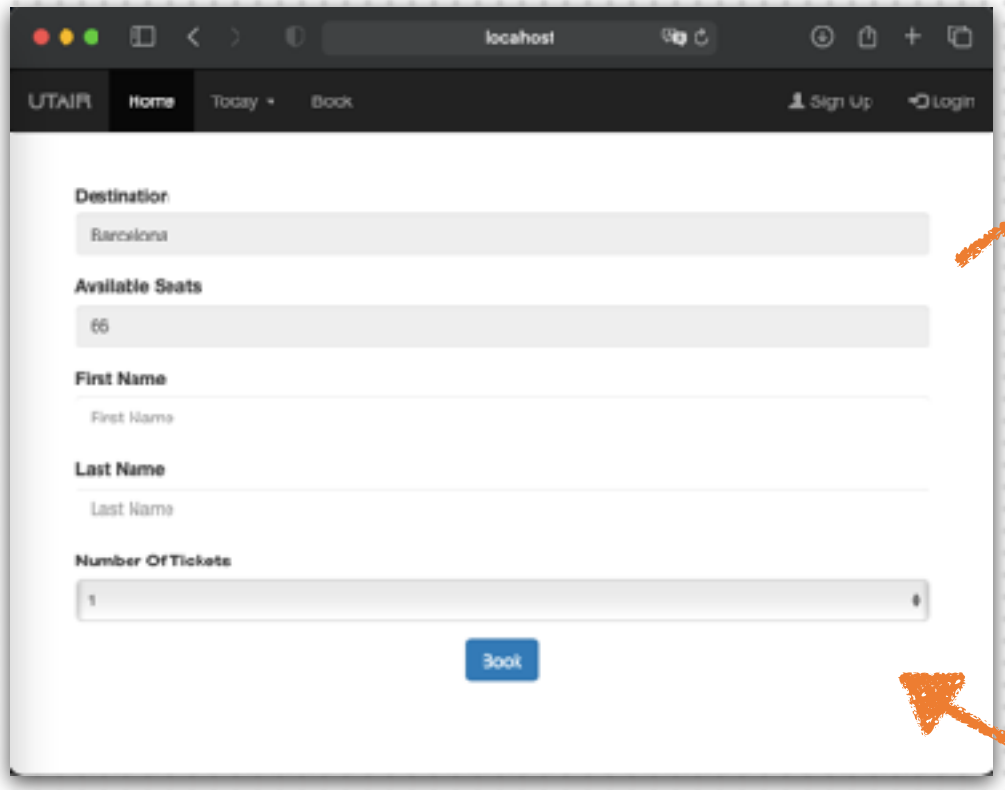
JSP Actions

- **Include:** <jsp:include page="inlcudedPage.jsp">
  - Includes the the output of includedPage.jsp into the page where this is included.
- **Forward:** <jsp:forward page="Forwarded.html">
  - Forwards the response to other web specification resources
- **setProperty:** <jsp:setProperty name="myBean" property="firstName" value="ali"/>
  - Passing value to other pages

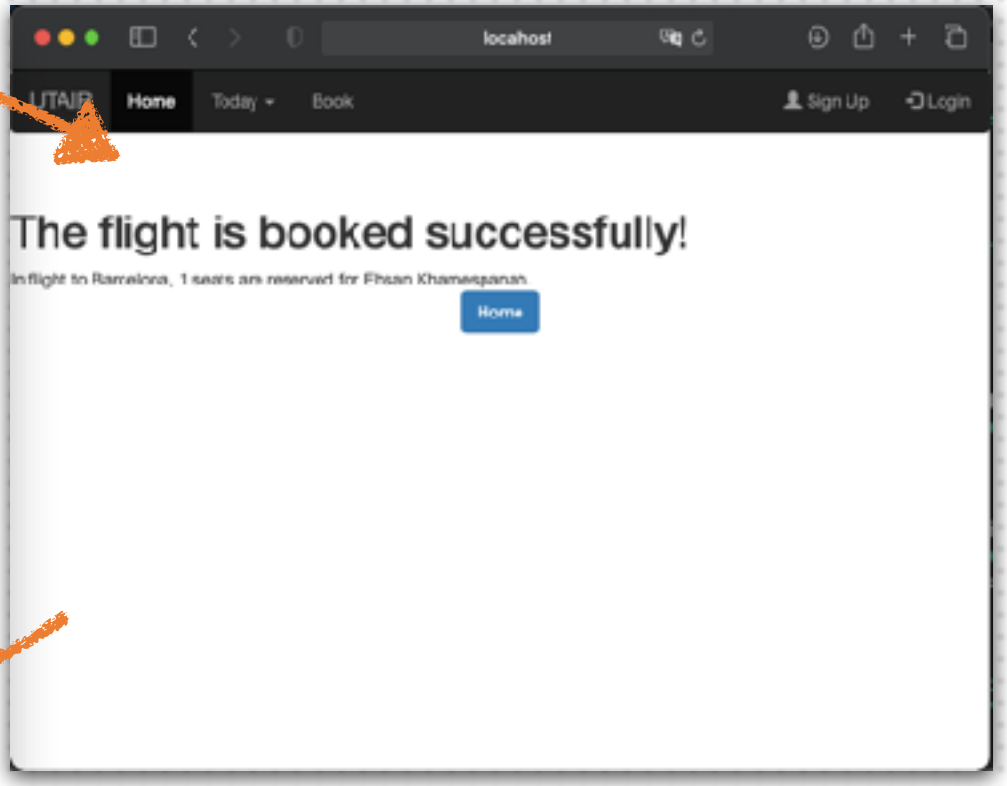
Demo



Validity of Input Data



Data of Booking Form



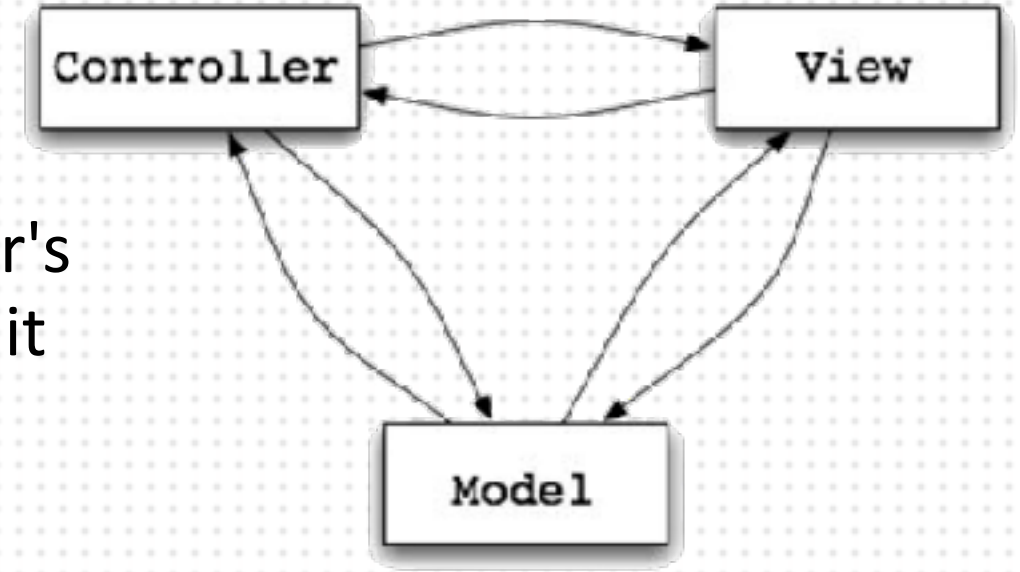
Incorrect Data Report

Think More!

- Different duties are mixed together
  - UI based activities
  - Navigation and forwarding strategies
  - Domain Logic and presentation
- Different concerns in each part
- Separation of duties?

MVC Pattern

- MVC is a design pattern that splits the user interface interaction into three distinct roles.
- It is a pattern that is all about the user interface. So it belongs in the user interface layer. Even the model!
  - Model stores the state of the presentation
  - The controller's job is to take the user's input and figure out what to do with it
  - View contains presentation details



The Main Responsibilities of the Controller

- Routing input request to the desired destination
- Input data validation
  - Email address is correct?
- Transforming output data
  - Transforming data to JSON
- Forward to the result page
  - Error page or valid target page

Demo